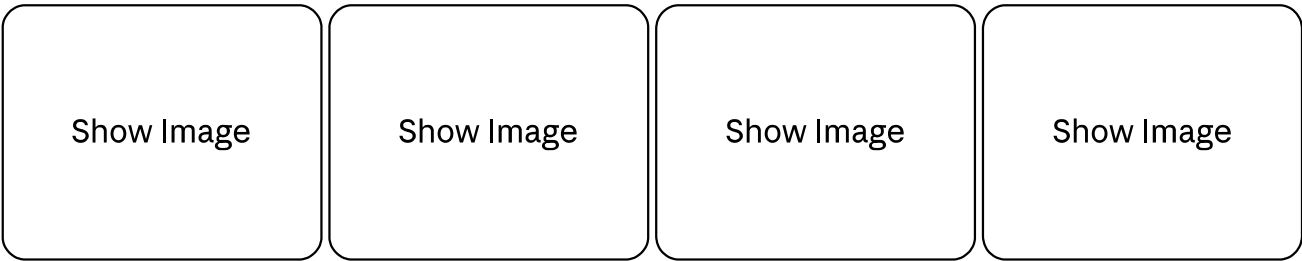


Code Risk Analyzer

Automated pull-request risk analysis for GitHub. Every PR is fetched, scanned through a multi-layer analysis pipeline, scored, reviewed by an AI agent, and commented back on the PR — with all results surfaced in a real-time dashboard.



Overview

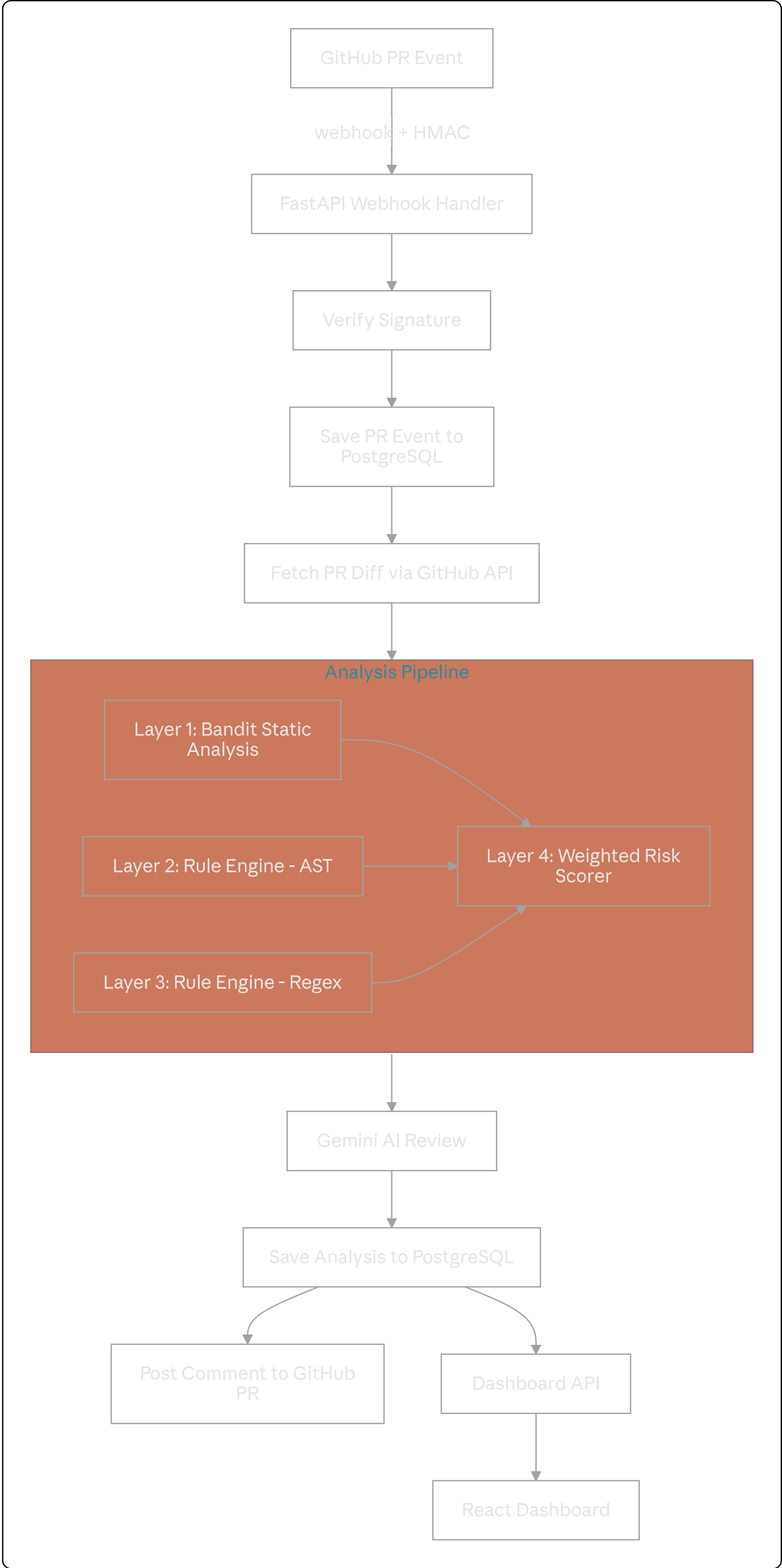
Code Risk Analyzer is a GitHub webhook-driven service that automatically reviews pull requests for security, maintainability, and architectural risk. When a PR is opened, the system pulls the diff, runs it through four independent analysis layers, computes a weighted risk score, generates a human-readable AI review, and posts a structured comment directly back to the pull request. A React dashboard provides aggregate visibility across all analyzed PRs.

The project was built to demonstrate an end-to-end backend system: async API design, an extensible analysis pipeline, third-party integrations (GitHub API, Gemini), relational data modeling, and a polished frontend.

Key Features

- **GitHub webhook integration** with HMAC-SHA256 signature verification
- **Multi-layer analysis pipeline** combining industry tooling and custom logic
- **Bandit** static security analysis for Python
- **Custom rule engine** using both AST traversal (structural checks) and regex (pattern checks)
- **Weighted risk scoring** across severity and rule categories, mapped to risk levels
- **AI review layer** (Google Gemini) that summarizes risk, explains issues, and recommends fixes
- **Automated PR comments** posted back to GitHub with a risk summary table and AI review
- **Dashboard API** exposing stats, risk distribution, top-risky PRs, and recent activity
- **React + Tailwind dashboard** with charts, drill-down detail pages, and rendered AI reviews

Architecture



Analysis Layers

Layer	Tool / Approach	Catches
1 — Static Analysis	Bandit	Known security anti-patterns (shell injection, weak hashing, hardcoded passwords)
2 — Rule Engine (AST)	Python <code>ast</code> module	Nested loops, oversized functions, bare excepts, <code>eval</code> / <code>exec</code> usage
3 — Rule Engine (Regex)	Pattern matching	Hardcoded secrets, SQL string concatenation, leftover TODOs, stray prints
4 — Risk Scoring	Weighted aggregation	Combines severity + rule weights into a single score and risk level

Tech Stack

Backend

- FastAPI (async) — webhook handling and dashboard API
- SQLAlchemy 2.0 (async) + asyncpg — ORM and PostgreSQL driver
- PostgreSQL — persistence for PR events and analysis results
- Bandit — static security analysis
- Google Gemini — AI review generation
- httpx — async GitHub API client
- Pydantic Settings — typed configuration

Frontend

- React + Vite
- Tailwind CSS
- Recharts — risk distribution charts
- React Router — navigation
- react-markdown — rendering AI reviews

Project Structure

```
code-risk-project/
├── Backend/
│   ├── main.py                # App entry, lifespan, router
│   ├── registration
│   │   └── backend/
│   │       ├── config.py      # Typed settings from .env
│   │       └── app/
│   │           ├── routes/
│   │           │   ├── webhook.py    # PR webhook receiver + pipeline
│   │           └── orchestration
│   │               ├── dashboard.py   # Dashboard API endpoints
│   │               ├── services/
│   │               │   ├── github_services.py # Signature verify, diff fetch,
│   │               │   └── comment post
│   │               ├── static_analysis.py # Bandit + pipeline coordination
│   │               ├── rule_base_analysis.py # AST + regex rule engine
│   │               ├── scorer.py      # Weighted risk scoring
│   │               ├── ai_review.py   # Gemini review + comment formatting
│   │               ├── database_service.py # Persistence helpers
│   │               └── dashboard_services.py # Dashboard queries
│   └── database/
│       ├── session.py          # Async engine + session factory
│       └── models/
```

```
|           └─ models.py           # SQLAlchemy models
└─ frontend/
    └─ src/
        ├── api/client.js          # Central API layer
        ├── components/           # Cards, charts, tables, badges
        ├── pages/                # Dashboard, AnalysisDetail
        └─ utils/format.js        # Colors, formatters
```

Getting Started

Prerequisites

- Python 3.12+
- Node.js 18+
- PostgreSQL 14+
- A GitHub personal access token (with `repo` / pull request + issues write access)
- A Google Gemini API key

1. Clone

```
bash

git clone https://github.com/GRushiBhargav/code-risk-analyzer.git
cd code-risk-analyzer
```

2. Backend setup

```
bash

cd Backend

# create + activate virtual environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# install dependencies
pip install -r requirements.txt
```

Create a `.env` file in the `Backend/` directory:

```
env

GITHUB_WEBHOOK_SECRET=your_webhook_secret
DATABASE_URL=postgresql+asyncpg://postgres:password@localhost:5432/coderisk
GITHUB_TOKEN=your_github_token
GEMINI_API_KEY=your_gemini_key
```

Create the database:

```
bash

psql -U postgres -c "CREATE DATABASE coderisk;"
```

Run the server (tables are created automatically on startup):

```
bash

uvicorn main:app --reload
```

The API is now available at `http://localhost:8000` — interactive docs at `http://localhost:8000/docs`.

3. Frontend setup

```
bash
```

```
cd frontend
npm install
npm run dev
```

The dashboard is available at `http://localhost:5173`.

4. Connect a GitHub webhook

In your test repository: **Settings** → **Webhooks** → **Add webhook**

- **Payload URL:** your public endpoint (use `ngrok` for local testing, e.g. `https://<id>.ngrok.io/webhook`)
- **Content type:** `application/json`
- **Secret:** same value as `GITHUB_WEBHOOK_SECRET`
- **Events:** Pull requests

Open a pull request and watch the analysis appear both as a PR comment and on the dashboard.

API Reference

Method	Endpoint	Description
POST	/webhook	Receives GitHub PR events and runs the pipeline
GET	/dashboard/stats	Top-level KPIs (PRs, analyses, avg score, findings)
GET	/dashboard/risk-distribution	Count of analyses per risk level
GET	/dashboard/top-risky?limit=	Highest-scoring PRs
GET	/dashboard/recent-activity?limit=	Latest PR events
GET	/dashboard/analyses?limit=&offset=	Paginated analysis list
GET	/dashboard/analyses/{id}	Full analysis detail incl. findings + AI review

Roadmap

- ☐ Alembic migrations for versioned schema changes
- ☐ Background task processing so webhooks return instantly
- ☐ Architectural pattern checks (SOLID violations, god classes, tight coupling)
- ☐ Vector retrieval over historical PRs for richer AI context
- ☐ Docker Compose + cloud deployment with CI/CD

License

MIT